

SQL

EEEP MANUEL ABDIAS EVANGELISTA
Prof. DAVI MAGALHÃES
BANCO DE DADOS 2023.2

SQL

A linguagem SQL (Structured Query Language) é uma linguagem de programação utilizada para **gerenciar e manipular bancos de dados relacionais**.



SQL

SQL é uma linguagem de consulta: usada principalmente para recuperar informações de bancos de dados, inserir novos dados, atualizar registros existentes e excluir informações.

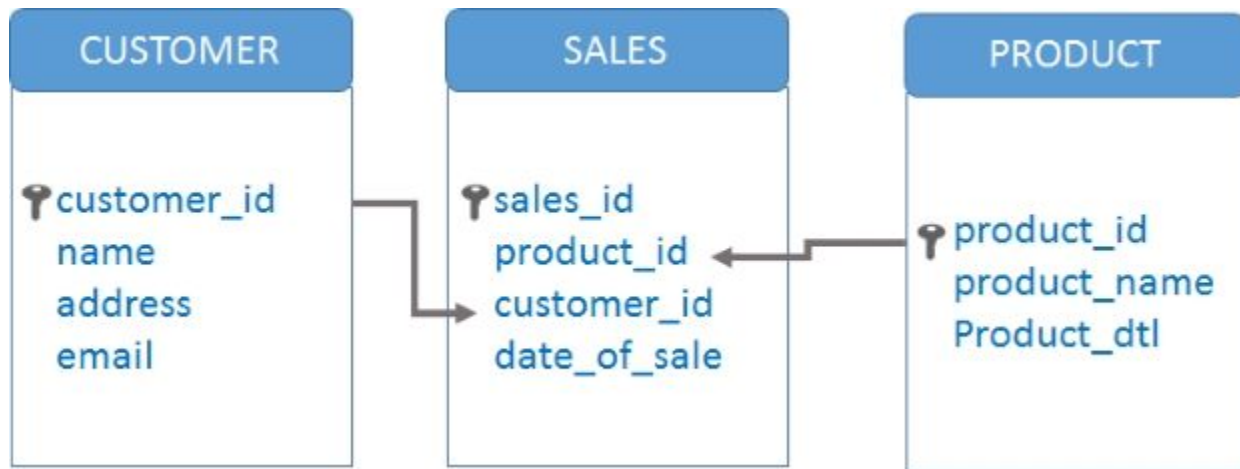
Diagram illustrating a database table structure. The table is labeled **Persons** and contains the following data:

Id	Name	SurName	Age
1	Jodie	Tucker	34
2	Jayden	Archer	56
3	Grace	Wheeler	18
4	Freddie	Humphries	56

An arrow labeled **Columns** points to the header row, and an arrow labeled **Rows** points to the data rows.

SQL

Bancos de dados relacionais: SQL é tipicamente associada a bancos de dados relacionais, que armazenam dados em tabelas interconectadas.



Banco de dados (minimundo)

Criar banco de dados:

```
CREATE DATABASE nome_do_minimundo;
```

Excluir banco de dados:

```
DROP DATABASE nome_do_minimundo;
```

EVITAR UTILIZAR!



Banco de dados (minimundo)

Mostrar bandos de dados disponíveis:

```
SHOW DATABASES;
```

Acessar um banco de dados para usá-lo:

```
USE nome_do_minimundo;
```

Relações (tabelas)

Mostrar tabelas do banco de dados selecionado:

```
SHOW TABLES;
```

Relações (tabelas)

Criar tabelas:

```
CREATE TABLE nome_da_tabela (  
    atributo TIPO,  
    atributo TIPO(X),  
    atributo TIPO(X)  
);
```

Relações (tabelas)

Criar tabelas, exemplo:

```
CREATE TABLE pessoa (  
    nome VARCHAR(100),  
    cpf INT,  
    data_nascimento DATE  
);
```

Relações (tabelas)

Mostrar colunas da tabela:

```
SHOW COLUMNS FROM persons;
```

Relações (tabelas)

Excluir a tabela (e seus dados):

```
DROP TABLE nome_da_tabela;
```

USAR COM CUIDADO!



Relações (tabelas)

Excluir **apenas os dados** de uma tabela:

```
TRUNCATE TABLE nome_da_tabela;
```

USAR COM CUIDADO!



Relações (tabelas)

Alterar tabelas (**adicionar**):

```
ALTER TABLE nome_da_tabela  
ADD nome_da_coluna TIPO;
```

Alterar tabelas (**remover**):

```
ALTER TABLE nome_da_tabela  
DROP COLUMN nome_da_coluna;
```

Relações (tabelas)

Renomear coluna:

```
ALTER TABLE nome_da_tabela
```

```
CHANGE COLUMN antigo_nome novo_nome TIPO;
```

Restrições

Criar tabela com restrições:

```
CREATE TABLE nome_da_tabela (  
    atributo TIPO RESTRICAO,  
    atributo TIPO(X) RESTRICAO,  
    atributo TIPO(X) RESTRICAO  
);
```

Restrições

Principais tipos:

Chave primária	PRIMARY KEY
Chave estrangeira	FOREIGN KEY
Sempre ter valor (não aceitar vazio)	NOT NULL
Valor único (não pode se repetir)	UNIQUE
Valor padrão	DEFAULT

Relações (tabelas)

Criar tabela com restrições, exemplo:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL  
);
```

Relações (tabelas)

Criar tabela com **chave primária**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL  
);
```

Relações (tabelas)

Criar tabela com **chave primária**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL,  
    PRIMARY KEY (cpf)  
);
```

Relações (tabelas)

Criar tabela com **chave primária composta**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL,  
    rg INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL  
);
```

Relações (tabelas)

Criar tabela com **chave primária composta**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL,  
    rg INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL,  
    CONSTRAINT PK_Identificar PRIMARY KEY (cpf, rg)  
);
```

Chave primária

Adicionar chave primária à tabela:

```
ALTER TABLE pessoa  
ADD PRIMARY KEY (cpf);
```

Remover chave primária da tabela:

```
ALTER TABLE pessoa  
DROP PRIMARY KEY;
```

Chave primária **auto incremental**

Criar tabela com chave primária com **valor automático sequencial**:

```
CREATE TABLE endereco (  
    cod INT NOT NULL PRIMARY KEY,  
    rua VARCHAR(100) NOT NULL,  
    cidade VARCHAR(100) NOT NULL  
);
```

Chave primária **auto incremental**

Criar tabela com chave primária com **valor automático sequencial**:

```
CREATE TABLE endereco (  
    cod INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
    rua VARCHAR(100) NOT NULL,  
    cidade VARCHAR(100) NOT NULL  
);
```

Relações (tabelas)

Criar tabela com **chave estrangeira (valor opcional)**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL,  
    cod_endereco INT,  
    FOREIGN KEY (cod_endereco) REFERENCES Endereco(cod)  
);
```

Relações (tabelas)

Criar tabela com **chave estrangeira (valor obrigatório)**:

```
CREATE TABLE pessoa (  
    cpf INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    data_nascimento DATE NOT NULL,  
    cod_endereco INT NOT NULL,  
    FOREIGN KEY (cod_endereco) REFERENCES Endereco(cod)  
);
```

Chave estrangeira

Adicionar chave estrangeira à tabela:

```
ALTER TABLE pessoa
```

```
ADD FOREIGN KEY (cod_endereco) REFERENCES Endereco(cod);
```

Valor padrão

Definir valor padrão para atributo:

```
CREATE TABLE endereco (  
    cod INT NOT NULL PRIMARY KEY,  
    rua VARCHAR(100) NOT NULL,  
    num INT NOT NULL,  
    bairro VARCHAR(100) NOT NULL,  
    cidade VARCHAR(50) DEFAULT 'Nova Russas',  
);
```

Valor padrão

Alterar atributo para definir valor padrão:

```
ALTER TABLE endereco
```

```
ALTER bairro SET DEFAULT 'Centro';
```

Contato

davimagal.sc@gmail.com